Universidad Nacional Autónoma de México

Ingeniería en ...

Nombre de Asignatura

FORMATO DE REPORTE

ALUMNO:
309075318

Grupo:
#
Semestre:
2024-II

México,CDMX. Febrero 2024

Índice

1. Introducción **2**

2. Marco Teórico **2**

3. Desarrollo **2**

3.1. Algoritmo: Carrucel de anuncios/Publicidad 2

3.1.1. Descripción de Funciones 3

3.1.2. Pruebas 6

4. Resultados **7**

5. Conclusiones **7**

1. Introducción

Este apartado debe abordar los siguientes puntos:

- **Planteamiento del problema.** Se hace una descripción del problema a resolver.
- **Motivación.** Se describe porqué es necesario dar solución al problema.
- **Objetivos.** Lo que se se espera obtener de darle solución al problema.

2. Marco Teórico

Son los conceptos aplicados, principalmente los vistos en clase, con los que se va a dar solución al problema, explicados de forma concreta, ya sea con su interpretación personal o bien con referencias bibliográficas sólidas. [1]

3. Desarrollo

3.1. Algoritmo: Carrucel de anuncios/Publicidad

1. Se crea una lista.
2. Se insertan varios anuncios con su ID, título y descripción.
3. Se muestran los anuncios al usuario en forma de carrusel.
4. Se le permite al usuario avanzar o retroceder entre los anuncios.
5. Se busca un anuncio por medio de su ID.
6. Se muestra un mensaje si el anuncio fue encontrado o no.
7. Se intenta eliminar un anuncio con un ID que no existe.
8. Se muestra la lista de anuncios actual.
9. Se elimina un anuncio con un ID existente.
10. Se vuelve a mostrar la lista para comprobar que el anuncio fue eliminado.
11. Por último se libera la memoria utilizada por la lista.

3.1.1. Descripción de Funciones

- Función **crearNodo**: La función recibe tres argumentos: un entero **id**, una cadena **titulo** y una cadena **descripcion**. Su funcionamiento se divide en los siguientes pasos:

  1. Se crea un nuevo nodo reservando memoria dinámicamente con la función **malloc**.
  2. Se verifica si la memoria fue asignada correctamente. Si no se pudo asignar, se imprime un mensaje de error y la función regresa **NULL**.
  3. Se asigna el valor de **id** al campo **nuevo->id**.
  4. Se copia la cadena **titulo** al campo **nuevo->titulo** con ayuda de la función **strcpy**.
  5. Se copia la cadena **descripcion** al campo **nuevo->descripcion**.
  6. Se inicializan los apuntadores **nuevo->anterior** y **nuevo->siguiente** en **NULL**.
  7. Finalmente, la función regresa el apuntador al nodo creado.

- Función **insertarAnuncio**: La función recibe cuatro argumentos: el apuntador doble a la lista, un entero **id**, una cadena **titulo** y una cadena **descripcion**. Su funcionamiento se divide en los siguientes pasos:

  1. Se crea un nuevo nodo llamando a la función **crearNodo**.
  2. Se verifica si el nodo fue creado correctamente. Si el nodo es **NULL**, la función termina.
  3. Si la lista está vacía, el nuevo nodo se enlaza a sí mismo, asignando **nuevo->siguiente = nuevo** y **nuevo->anterior = nuevo**, y el apuntador ***cabeza** se actualiza para apuntar al nuevo nodo.
  4. Si la lista no está vacía, se obtiene el último nodo de la lista mediante **(*cabeza)->anterior**.
  5. Se enlaza el nuevo nodo al final de la lista, asignando **nuevo->siguiente = *cabeza** y **nuevo->anterior = ultimo**.
  6. Después, se actualizan los enlaces del último nodo y del primero, asignando **ultimo->siguiente = nuevo** y **(*cabeza)->anterior = nuevo**.

- Función **buscarAnuncio**: La función recibe dos argumentos: el apuntador al inicio de la lista y un entero **id**. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, la función regresa **NULL**.
  2. Se utiliza un apuntador auxiliar **actual** para comenzar el recorrido desde el primer nodo de la lista.
  3. Se recorre la lista doble circular con una estructura **do-while**.

4. En cada nodo se compara el valor de **actual->id** con el **id** recibido.
5. Si se encuentra coincidencia, la función regresa el apuntador al nodo correspondiente.
6. Si se completa el recorrido y no se encuentra el anuncio, la función regresa **NULL**.

- Función **eliminarAnuncio**: La función recibe dos argumentos: el apuntador doble a la lista y un entero **id**. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, se imprime un mensaje y la función termina.
  2. Se busca el nodo que contiene el **id** recibido llamando a la función **buscarAnuncio**.
  3. Si el nodo no se encuentra, se imprime un mensaje indicando que el id no existe en la lista y la función termina.
  4. Si la lista tiene un solo nodo, se libera su memoria con **free**, se actualiza ***cabeza** a **NULL** y la función termina.
  5. Si la lista tiene más de un nodo, se actualizan los enlaces de los nodos vecino, asignando **eliminarNodo->anterior->siguiente = eliminarNodo->siguiente** y **eliminarNodo->siguiente->anterior = eliminarNodo->anterior**.
  6. Si el nodo eliminado corresponde a la cabeza de la lista, el apuntador ***cabeza** se actualiza al siguiente nodo.
  7. Se obtiene el último nodo mediante **(*cabeza)->anterior** y se refuerza la estructura circular, actualizando el enlace entre el último y el primero.
  8. Finalmente, se libera la memoria del nodo eliminado con **free()** y se imprime un mensaje de confirmación.

- Función **mostrarAnuncioActual**: La función recibe como argumento un apuntador al nodo actual. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si el apuntador recibido es **NULL**. Si lo es, se imprime un mensaje indicando que no hay anuncios y la función termina.
  2. Si existe un nodo válido, se imprimen en pantalla los datos del anuncio actual: **id**, **titulo** y **descripcion**.

- Función **siguienteAnuncio**: La función recibe como argumento un apuntador al nodo actual. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si el apuntador recibido es diferente de **NULL**.
  2. Si existe un nodo válido, el apuntador avanza al siguiente nodo mediante **actual = actual->siguiente**.

3. Después se llama a la función **mostrarAnuncioActual** para mostrar la información del nuevo anuncio actual.
4. Si no hay anuncios, se imprime un mensaje indicando que no hay anuncios que mostrar.

- Función **retrocederAnuncio**: La función recibe como argumento un apuntador al nodo actual. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si el apuntador recibido es diferente de **NULL**.
  2. Si existe un nodo válido, el apuntador retrocede al nodo anterior mediante **actual** = **actual->anterior**.
  3. Después se llama a la función **mostrarAnuncioActual** para mostrar la información del anuncio actual.
  4. Si no hay anuncios, se imprime un mensaje indicando que no hay anuncios que mostrar.

- Función **mostrarAnuncios**: La función recibe como argumento el apuntador al inicio de la lista. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, se imprime un mensaje y la función termina.
  2. Se utiliza un apuntador auxiliar **actual** para comenzar en el primer nodo de la lista.
  3. Dentro de un ciclo, se llama a la función **mostrarAnuncioActual** para mostrar el anuncio actual.
  4. Se presenta un menú con tres opciones: avanzar al anuncio siguiente, regresar al anuncio anterior o dejar de mostrar anuncios.
  5. Dependiendo de la opción elegida por el usuario, el apuntador **actual** se actualiza al nodo siguiente o anterior.
  6. Si el usuario elige salir, la función termina.

- Función **mostrarLista**: La función recibe como argumento el apuntador al inicio de la lista doble circular. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, se imprime un mensaje y la función termina.
  2. Se utiliza un apuntador auxiliar **actual** para recorrer la lista desde el primer nodo.
  3. Mediante una estructura **do-while**, se imprimen los datos de cada anuncio: **id**, **titulo** y **descripcion**.
  4. El recorrido continúa avanzando con **actual** = **actual->siguiente** hasta regresar nuevamente al nodo inicial.

- Función **liberarLista**: La función recibe como argumento el apuntador doble a la lista. Su propósito es liberar la memoria dinámica ocupada por todos los nodos. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, la función termina.
  2. Se utiliza un apuntador auxiliar **actual** para comenzar el recorrido desde el nodo siguiente al primero.
  3. Mientras no se regrese al nodo inicial, se guarda el siguiente nodo en un apuntador temporal, se libera el nodo actual con **free()** y se continúa con el siguiente.
  4. Finalmente, se libera también el primer nodo de la lista.
  5. El apuntador ***cabeza** se actualiza a **NULL**.

3.1.2. Pruebas

ID	Título	Descripción
1234	Tacos	Ricos tacos de cabeza
1235	Tortas	Ricas tortas
1236	Aguas	Aguas frescas de horchata y jamaica
1237	Pumagua	Pumagua la bebida oficial de los paristas
1238	Mcdonals	Me encanta

Tabla 1: Inserción de anuncios en la lista doble circular

ID buscado	Resultado esperado	Resultado obtenido
1236	Nodo encontrado	Se encontró el id 1236

Tabla 2: Prueba de la función buscarAnuncio

ID eliminado	Resultado esperado	Resultado obtenido
1239	No encontrado	El id 1239 no se encontró

Tabla 3: Prueba de eliminación de un ID inexistente

ID	Título	Descripción
1234	Tacos	Ricos tacos de cabeza
1235	Tortas	Ricas tortas
1236	Aguas	Aguas frescas de horchata y jamaica
1237	Pumagua	Pumagua la bebida oficial de los paristas
1238	Mcdonals	Me encanta

Tabla 4: Lista después de intentar eliminar un ID inexistente

ID eliminado	Resultado esperado	Resultado obtenido
1236	Nodo eliminado	Se eliminó el id 1236

Tabla 5: Prueba de eliminación de un anuncio existente

ID	Título	Descripción
1234	Tacos	Ricos tacos de cabeza
1235	Tortas	Ricas tortas
1237	Pumagua	Pumagua la bebida oficial de los paristas
1238	Mcdonals	Me encanta

Tabla 6: Lista después de eliminar el anuncio con ID 1236

4. Resultados

Mediante capturas de pantalla y una breve descripción seguida de la captura se presentan los resultados finales de su aplicación.

5. Conclusiones

Se presenta un análisis de los resultados obtenidos, donde se destaca la importancia de la aplicación de los conceptos teóricos para resolver el problema. Es importante resaltar lo siguiente:

- No es describir si les gustó la actividad o no.
- No es decir que se obtuvo de la práctica.
- No es describir lo que fue difícil.

Referencias

[1] Brian W. Kernighan y Dennis M. Ritchie. *The C Programming Language*. 2nd. Prentice Hall Professional Technical Reference, 1988. ISBN: 0131103709.